

Fases 3 y 4 — qué quedó hecho, con qué se probó y qué falta

SPACE OS · Plan de Instancias Soberanas v3 · corte al 2026-08-26 Reporte de avance para dirección de proyecto.

Rama de trabajo	feat/servidor-padre-instancias
Documento de autoridad	docs/Plan_Instancias_Soberanas_v3.md §FASE 3 (:917-1228) y §FASE 4 (:1230-1434)
Periodo cubierto	12 – 26 de agosto de 2026
Commits sobre main	235
Expediente de origen	docs/evidencias/fase-3-y-4.md

Cubre las 14 tareas de las Fases 3 y 4, las seis decisiones de arquitectura que se tomaron sobre la demostración y el dominio space-os.io , y el único punto que hoy impide dar la Fase 4 por cerrada.

1 · Resumen

	Estado	Detalle
Fase 3	7 de 9	Las dos restantes esperan una decisión externa, no trabajo de programación
Fase 4	3 de 4	De las tareas que conservan objeto; la quinta quedó anulada por decisión de producto
Para cerrar la Fase 4	0 acciones	Ninguna tarea humana pendiente. Queda una decisión de producto abierta (§4)
Bloqueo externo	1	El registry de imágenes (TH-P4), abierto desde el 17/08. Detiene seis tareas de tres fases

El estado en una frase: el motor de actualización de las instancias está escrito, probado y ya corrió contra servidores de verdad; la demostración está montada y funcionando; el sistema tiene por fin dominio propio, certificado válido y acceso con Google; y la Fase 4 no tiene ya ninguna acción humana pendiente.

Lo que sí queda abierto en la Fase 4 es una decisión de producto: qué máquina va a servir el nombre público de la demostración. Está deliberadamente sin decidir, y §4 explica por qué.

Lo que sí está detenido —el ensayo del ciclo completo de actualización— no depende de programar nada: depende de que exista el almacén donde se publican las versiones del producto. Esa decisión lleva abierta desde el 17 de agosto y es, con diferencia, el punto de mayor apalancamiento del plan.

Cómo leer este reporte

Cada afirmación lleva su ancla: un hash de commit, una referencia a archivo y línea, o la salida literal del comando que alguien corrió. Lo que se reporta sin haberlo medido va marcado como reporte y no como hecho — hay dos casos en todo el documento y los dos están señalados donde aparecen.

2 · Fase 3 · El motor de actualización de las instancias

Es la pieza central del modelo: cada instancia de cliente se actualiza sola, jalando su versión, respaldando su base antes de tocarla y devolviéndose sola si algo sale mal. Nadie entra por SSH a compilar en el servidor de un cliente. Es también la fase más grande del plan y la que más auditorías consumió.

Tarea	Qué hace	Estado	Evidencia
F3.1	Cada instancia lleva registro de las migraciones que ya aplicó	✅ Cerrada + probada en servidor	6cb16d4 · 14/08. Backfill de 65 filas, con tres exclusiones deliberadas
F3.2	El runner aplica lo pendiente, en el orden que de verdad funciona	✅ Cerrada + probada en servidor	d31a7b8 · 17/08. Tres ciclos; el primero auditado en rojo y corregido
F3.3	Reescribir una migración ya aplicada detiene la actualización	✅ Cerrada	dc6df52 · 17/08. Aborta con salida 3 nombrando el archivo y los dos checksums
F3.4	update.sh : jala el canal, respalda, migra y se devuelve solo si falla	✅ Ensayada, 9 puntos demostrados	acbbe0b → 8151772 → 2633bcb · 17–18/08
F3.5	Ensayo completo del ciclo release → update → rollback en DEMO	🔴 Bloqueada · TH-P4	Sin registry no hay canal beta del que jalar
F3.6	Retirar del repositorio el despliegue por SSH	🔴 Bloqueada · deliberado	Medido el 24/08: 3 coincidencias en deploy.yml (:68 , :171 , :185)
F3.7	El respaldo sale del droplet y sobrevive a su muerte	✅ Cerrada	f369b4c · 18/08. Credenciales fuera de argv ; chmod 600 antes del secreto
F3.8	El pull reintenta con backoff; la migración no reintenta nunca	✅ Cerrada	84c6c20 · 18/08. Medido: 1 s + 5 s + 30 s = 36,9 s reales, salida 1, estado vacío
F3.9	El log de la actualización se lee sin entrar al servidor del cliente	✅ Cerrada tras 4 ciclos	a490dd3 → 8f81c3e → 3872d61 · 19–20/08

2.1 · Lo que ganó esta fase y no estaba planeado

Hasta el 24 de agosto, el aplicador de migraciones solo se había probado contra bases desechables en la máquina de desarrollo. Desde entonces ha corrido **tres veces contra servidores reales**, y eso convierte a F3.1 y F3.2 de «probadas en simulación» a «probadas en producción».

```
$ migrar.mjs --instalacion-nueva          # base spaces_demo, 2026-08-24
--instalacion-nueva verificada: ninguna de las 11 tablas que solo crean las
migraciones existe en esta base.
71 aplicadas, 1 de datos pendientes.

$ migrar.mjs                             # segunda corrida, sin cambios
0 aplicadas, 1 de datos pendientes.
```

Ese **0 aplicadas** de la segunda corrida es la demostración de que el registro funciona: la instancia sabe lo que ya hizo y no lo repite. Y la guarda de

--instalacion-nueva se verificó a sí misma antes de tocar nada, que es exactamente la situación donde importa: una base a punto de recibir 71 migraciones. Después se aplicó también a **spaces_prod** (25/08, 1 aplicada) y las dos bases del servidor quedaron en 72.

Una corrección de cifras que conviene tener clara: son 71, no 72

Una emisión anterior del expediente daba «esperado: 72» y estaba mal. La migración **20260731_calendario_meses_cortos.sql** lleva la marca **-- @tipo: datos** y el runner la omite salvo que se le pida expresamente (**migrar.mjs:717-721**), igual que hacía el despliegue anterior: las migraciones de datos **reescriben filas y no se deshacen solas**.

Eran dos números distintos —archivos en disco y migraciones aplicables— y se estaban comparando como si fueran uno. El runner nombra en voz alta la que omite, a propósito: «una migración que no se aplica y que nadie menciona es una que se olvida» (**migrar.mjs:713-715**).

2.2 · Lo que costó de verdad: F3.9, cuatro ciclos y un cambio de enfoque

La tarea pedía que el registro de una actualización fallida se pudiera leer sin entrar al servidor del cliente. El problema apareció al medir antes de diseñar: **subir ese archivo tal cual habría incumplido el propio criterio de la tarea**, porque ya arrastraba salida cruda del aplicador de migraciones, de las herramientas de respaldo y de los registros de la aplicación.

La solución no fue añadir un filtro, sino ****separar lo que escribe el script de lo que escriben sus herramientas****. Viaja un archivo publicable; se queda el crudo. Sin lista de palabras prohibidas y sin expresiones regulares, a propósito: un filtro se olvida de un caso y nadie se entera.

El invariante quedó escrito en el propio script (`update.sh:905-911`):

```
«Una lista negra sobre un espacio de nombres que se decodifica
no se puede demostrar completa. Siempre queda otra codificación.
Reconstruir la conexión sí se puede demostrar.»
```

La decisión de fondo fue arreglar en el origen: las herramientas de respaldo dejaron de recibir la dirección de la base como una URL —donde la contraseña viaja dentro— y pasaron a recibir **cuatro parámetros sueltos**. Recorrido de la tarea:

`d540833` (rojo) → `70b8cc5` (ámbar) → `6fb93ec` (rojo) → `a490dd3`, cerrada con
`8f81c3e` y endurecida por `3872d61`.

2.3 · F3.6 no se ha hecho, y es correcto que no se haya hecho

Su criterio dice que no debe quedar en el repositorio ningún camino que entre por SSH al servidor de una instancia a compilar. Se midió con el comando exacto que el plan indica y **el criterio no se cumple**: tres coincidencias en `deploy.yml`.

No es un defecto pendiente de arreglar — es una tarea que ****no debe ejecutarse todavía****, y el plan lo marca como riesgo alto (`plan:1126-1129`): mientras el canal de actualizaciones no esté probado, ese despliegue es el único mecanismo que existe. Retirarlo antes de tiempo dejaría al proyecto sin forma de desplegar nada.

3 · Fase 4 · Separar la demostración de producción

La fase existe para cerrar un riesgo concreto y con nombre: ***«demo pública = producción»***. Hasta julio, el sitio que se enseñaba a clientes y el que guardaba los datos eran la misma máquina y la misma base. El riesgo se cerró, pero por un camino distinto al que el plan imaginaba — y eso es lo que explica el apartado 4.

Tarea	Qué pedía	Estado	Evidencia
F4.1	Censar el servidor viejo antes de separar nada	✅ Cerrada · 25/08	<code>docs/evidencias/f4-1-censo-resultado.md</code> . Todo de solo lectura
F4.2	Base propia para la demostración, sin una fila de ningún cliente	✅ Cumplida	Los tres criterios, medidos con <code>psql</code> contra la base real
F4.3	Dominio y certificado propios para la demostración	⌚ Pendiente · ADR 0021	Condicionada a decidir qué máquina sirve el nombre
F4.4	Proceso propio, datos de juguete y banderas correctas	✅ Cumplida	Proceso medido con <code>systemctl</code> ; banderas medidas por HTTP
F4.5	Dejar por escrito que el riesgo dejó de ser cierto	🟡 2 de 4 criterios	Uno quedó sin objeto; otro se cierra borrando un registro DNS

3.1 · F4.1 — el censo, que además desmontó una premisa falsa

El 24 de agosto se concluyó que se había perdido el acceso al servidor viejo, y sobre esa conclusión se levantaron una decisión de arquitectura, una enmienda al plan y dos tareas declaradas «imposibles». El 25 se entró sin dificultad y se completó el censo entero. La premisa era falsa; las cuatro cosas se revisaron.

Dato censado	Valor medido
Identidad	PIXELED-ubuntu-s-2vcpu-4gb-nyc3 · 209.97.146.136
Commit desplegado	504b4fc (11/08), presente en main — la condición de parada del plan no se dispara
Proceso	online , 13 días de servicio, corriendo como emiliano y no como root
Certificado	demo.space-os.io , vence el 2026-10-26
Organizaciones	rgb , telcel , g500 , eyro , demo-owner — todas de julio
Prueba de vida real	POST /api/auth/login/ → 401: esa máquina sí habla con su base

3.2 · F4.2 — la base de la demostración, medida contra sus criterios

```
spaces_app|f|f    <- ni superusuario, ni puede saltarse el aislamiento
0                 <- cero organizaciones antes de sembrar
72                <- migraciones aplicadas, igual que la base de produccion
```

Los dos primeros valores son el criterio literal de la tarea: la base de la demostración no contiene ni una fila de ningún cliente, y el usuario con el que la aplicación se conecta no tiene privilegios para saltarse el aislamiento entre organizaciones.

3.3 · F4.4 — proceso separado, banderas cerradas, alta verificada

La demostración corre como **un proceso distinto, con su propio usuario del sistema**. Esto no es un detalle de implementación: es la separación de la que depende toda la decisión de que la demostración comparta máquina con el plano de control.

```
$ systemctl show spaces-demo -p MainPID --value | xargs -r ps -o user=,pid=,cmd= -p
demo      262995 next-server (v14.2.29)

$ curl -s http://127.0.0.1:3001/spaces-dooh/api/auth/metodos/
{"google":false,"autoregistro":false}

$ curl ... http://127.0.0.1:3001/spaces-dooh/login/
login 200

$ bootstrap-auth.mjs
OK · usuarios: 1 · organizacion: demo
Dueno: emistreg@gmail.com
```

El identificador de la organización es demo y no rgb a propósito: uno de los criterios de cierre compara los identificadores de las dos bases y exige que **no compartan ninguno**. Y el acceso con Google va apagado en la demostración por una razón concreta: encendido, habría heredado la configuración del servidor padre y mandado al visitante allí.

Lo único que en esta fase se reporta y no se midió

La siembra de inventario de juguete (`20260824_semilla_demo.sql`) **no tiene salida capturada**. Se reporta como corrida y el resultado esperado es `demo · 2 arrendadores · 6 pantallas · 5 disponibles` . Queda anotado como **reporte, no como medición**, y se cierra pegando esa línea la próxima vez que alguien entre al servidor.

3.4 · F4.5 — el cierre del riesgo, criterio por criterio

#	Criterio	Estado
1	La demostración resuelve a su propio servidor	✅ Cumplido — resuelve a <code>209.97.146.136</code>
2	Las dos bases no comparten ninguna organización	✅ Cumplido — <code>demo</code> frente a <code>rgb</code>
3	La máquina vieja ya no sirve ese nombre	❌ Retirado (ADR 0021) — el nombre se conserva a propósito
4	La demostración está suscrita al canal de pruebas	⬢ Desviación declarada — bloqueado por TH-P4

4 · Los cambios acordados: la demostración y `space-os.io`

Aquí está el cambio de fondo que un reporte de tareas no explica por sí solo. La decisión sobre dónde vive la demostración **cambió tres veces en dos días**, y cada giro tuvo una causa distinta. Está escrito así, y no como una sola decisión limpia, para que no se lea como indecisión: dos de los tres giros vinieron de que apareció información nueva, y el tercero de una decisión de producto.

24 de agosto · ADR 0015 — la demostración pasa a vivir dentro del servidor padre

Se acepta un precio y se escribe: el riesgo de la Fase 4 no se cierra, *se transforma* — deja de ser «demo pública = producción» y pasa a ser «demo pública = plano de control».

Causa: se creyó perdido el acceso al servidor viejo. **Superada.**

25 de agosto · ADR 0016 — la demostración se queda en su propio servidor

Reutilizar la máquina cerraba el riesgo de verdad y costaba menos trabajo.

Causa: el censo de F4.1 demostró que el acceso **nunca se perdió**. **Superada.**

25 de agosto · ADR 0017 — todo se concentra en el servidor padre · VIGENTE

El servidor padre (`137.184.107.53`) es la única máquina del modelo. La máquina vieja **no forma parte de él**: no se le añaden mejoras, no se le migra nada y no se cuenta con ella para nada del plan.

Causa: decisión de producto, no técnica. Mantener viva una máquina montada a mano en julio es mantener una excepción permanente al modelo que el plan entero intenta construir.

25 de agosto · ADR 0019 — la demostración arranca como servicio del sistema · VIGENTE

Al montarla, el gestor de procesos habitual resultó **inalcanzable** para el usuario `demo` : está instalado bajo el directorio de root, que es privado. Se descartó de plano la salida fácil —arrancarla como root— porque la separación por usuario es la *única* mitigación real del ADR 0017; quitarla dejaba esa decisión sin su fundamento.

Al escribirlo aparecieron dos defectos que nunca habían dado señal, entre ellos una configuración que habría hecho a la demostración pelearse por el puerto del servidor principal.

26 de agosto · ADR 0020 — no hay demostración pública · VIGENTE

`demo.space-os.io` queda **abandonado**: no se le mueve el DNS, no se le emite certificado y su registro se retira. `space-os.io` es lo oficial y también donde se prueba, y **la demostración de cara a cliente pasa a ser el producto funcionando con una o más instancias

hijas** — es decir, lo que produce la Fase 5. El proceso interno se queda como banco de pruebas, sin nombre público.

Causa: `demo.space-os.io` era la muestra de «cómo van a ser las instancias hijas» cuando todavía no existía ninguna. Una demostración que enseña el producto con instancias de verdad vale más que un sitio aparte que las imita.

26 de agosto · ADR 0021 — el nombre de la demostración se queda · VIGENTE

Revierte al ADR 0020, escrito el día anterior. `demo.space-os.io` **no se retira**: es el nombre público donde se enseñarán las instancias hijas cuando existan. Su registro DNS **no se borra**, y la tarjeta que pedía borrarlo queda **cancelada**.

Causa: el 0020 razonaba que ese nombre había nacido para *imitar* instancias hijas que todavía no existían, y concluía que al llegar las de verdad el nombre sobraba. **La conclusión correcta es la contraria:** cuando existan las hijas, ese nombre es exactamente donde se enseñan.

Lo que el ADR 0021 deja escrito COMO NO DECIDIDO

Se escribe aparte a propósito, porque tres de las cuatro decisiones anteriores sobre la demostración se revirtieron en cuatro días, y buena parte del coste vino de rellenar los huecos por deducción en vez de dejarlos marcados.

Qué máquina sirve `demo.space-os.io` a partir de ahora no está decidido. Hoy apunta a la máquina de julio. Las opciones abiertas son dejarlo ahí hasta que exista la primera instancia hija, o moverlo a esa instancia cuando nazca. De eso cuelgan, también sin resolver, si se le emite certificado y con qué método, y qué pasa con el certificado actual de esa máquina, que **vence el 26 de octubre**.

Y una condición aceptada, que ya no es una tarea

Mientras el nombre apunte a la máquina de julio, esa máquina **sigue sirviendo un sitio público** con cinco organizaciones dentro. Con el ADR 0021 eso deja de ser una tarea pendiente y pasa a ser una **condición aceptada**. Queda escrito una vez para que el estado del proyecto no mienta — no como recordatorio ni como objeción.

Lo que esto le ahorra a dirección respecto de lo que decía ayer

La emisión anterior de este reporte advertía de que **no habría dónde enseñar el producto hasta que existiera la primera instancia hija**. Con el 0021 esa advertencia **decae**: el nombre de la demostración se conserva, así que no se pierde el sitio donde enseñar. Lo que cambia es *qué* se enseña allí — el producto real corriendo como una instancia hija, no una imitación.

4.1 · Lo que sí se ganó: `space-os.io` está en pie y sirve el stack completo

El 25 de agosto el servidor padre demostró por primera vez la cadena entera —dominio, cifrado, servidor web, aplicación y base de datos— medida **por el nombre público y desde fuera**, no contra sí mismo.

```
raiz      302    <- el servidor web redirige
login     200    <- la aplicacion responde
login-post 401    <- LA BASE CONTESTA, a traves de TLS + servidor web + app
CN = space-os.io    notAfter = Nov 23 18:48:59 2026 GMT
```

Esa tercera línea es la importante: un `401` significa que la aplicación buscó en la base de datos de verdad y no encontró al usuario. Por la mañana esa misma cadena daba `500` y no había certificado.

Pieza	Estado al 26/08
Certificado	<code>space-os.io</code> hasta el 23 de noviembre, con renovación automática configurada
Emisión	Por servidor propio temporal — la configuración que corría no tenía hueco para el método habitual
Servidor web	Configuración enlazada desde el repositorio , no pegada a mano. La provisional se retiró
Cloudflare	Proxy en gris, decidido el 25/08
Acceso con Google	Funciona de punta a punta desde el navegador

Una dependencia permanente que se evitó por el camino

El camino original exigía un token de Cloudflare para renovar el certificado. Ese token, al caducar, **habría matado el sitio en silencio 90 días después** — un fallo que solo se detecta si alguien se acuerda de comprobarlo cada dos meses. Al invertir el orden de dos pasos, la renovación quedó automática y sin nada que recordar.

El certificado de la demostración se intentó cinco veces el 25/08 por el camino difícil y las cinco fallaron por la misma razón; con el ADR 0020 ese trabajo **desapareció entero**: no es que se resolviera, es que dejó de hacer falta.

4.2 · Un cambio de producto que salió de aquí, y se nota desde la aplicación

****ADR 0018** — quien entra con Google ya puede ponerse contraseña sin conocer la anterior.** Al crear la cuenta de un responsable, el sistema genera una contraseña temporal y la enseña una sola vez. Si esa persona entraba con Google y la temporal se había perdido, quedaba **encerrada**: la pantalla le pedía un dato que nadie tenía, y este servidor no envía correos de recuperación. Construido y verificado en producción el mismo día. Sigue haciendo falta la contraseña anterior para todo lo demás — la facilidad es de un solo uso por persona y desaparece en cuanto se usa.

Al construirlo aparecieron tres defectos, y el tercero es el que vale la pena contar: la regla existía **solo en el servidor**, y la pantalla cortaba el envío antes de llamarla — estaba implementada e inalcanzable. Y una consulta leía una tabla protegida por el aislamiento sin el contexto de la organización, con lo que devolvía vacío en silencio. Es la **tercera vez** que ese mismo modo de fallo aparece en el proyecto; las dieciséis pruebas unitarias que cubrían esa zona no podían verlo, porque simulan la base y **una simulación no tiene políticas de aislamiento**.

5 · Hallazgo transversal: cuatro cosas rotas y ninguna daba señal

Es el resultado más valioso de estas dos fases —más que la lista de tareas cerradas— porque cambia cómo hay que verificar los servidores de la Fase 5, que es la que viene.

El servidor padre pasó ****cuatro días sirviendo una pantalla de acceso que no podía autenticar a nadie****. Le faltaba la dirección de su base de datos, y el código —en vez de negarse a arrancar— se cae a un valor por omisión que apunta al ordenador de un programador. La aplicación arrancaba, pintaba la pantalla y devolvía `200`.

Cinco comprobaciones daban verde sobre un sistema roto

```
raiz 302 · login 200 · signup 503 · nginx -t ok · metodos google:true
```

Las cinco pasaban con la base caída y con el acceso con Google roto. También pasaba el humo del propio procedimiento de instalación. **Lo único que encuentra esta clase de fallo es ejercer la función de verdad.**

Qué estaba roto	Desde	Cómo se encontró
Sin dirección de base de datos	21/08	Un <code>POST</code> de acceso, no un <code>GET</code> de página
Identificador de Google sin su primer carácter	21/08	Pulsando el botón
Dirección de retorno apuntando al ordenador de un programador	21/08	Completando el flujo entero
Base de producción sin la última migración	24/08	Contando, no mirando

El del identificador de Google es el más instructivo: **le faltaba un carácter** —se lo comió la consola web al pegar la configuración— y nada lo delata. La aplicación arranca, el botón aparece, la comprobación de métodos dice que Google está disponible. Solo falla cuando alguien intenta entrar.

La consecuencia para la Fase 5, que hay que escribir antes de llegar

La comprobación que se correrá en cada servidor nuevo **hereda el mismo agujero** si se limita a mirar códigos de respuesta. Tiene que incluir **una petición que toque la base**: un intento de acceso con credenciales inexistentes, esperando `401`. Un `500` ahí significa que la instancia nació rota. Un `200` en la pantalla de acceso no significa nada.

6 · Control de calidad: qué se corrió y con qué resultado

Suite	Cifra medida	Fecha
Pruebas unitarias	858 en 79 archivos	25/08
Pruebas de integración	20 archivos · 216 pruebas · 1 saltada	25/08
Verificación de tipos	Limpia	25/08
Prueba de aislamiento entre organizaciones	Pasa sin tocarse — es el invariante que el plan exige	25/08
Arnés del actualizador	102 escenarios · 664 comprobaciones · 0 rojas (nació con 28 · 101)	Cierre de Fase 3
Migraciones	73 archivos · 72 aplicables	25/08

```
cd apps/web && npm run typecheck && npm test && npm run build && npm run test:e2e
```

Una trampa del entorno que costó un diagnóstico

Las pruebas de integración exigen una compilación hecha antes, porque el arnés reutiliza lo que encuentre. Sin compilación fallan las veinte en falso y tardan diez minutos en hacerlo — un rojo que no dice nada del código. Y la variante que costó un diagnóstico el 25/08: una compilación vieja también engaña.

7 · Lo que falta: una decisión, un bloqueo y seis tareas abiertas

7.1 · La decisión abierta de la Fase 4

⚠️ Cancelado el 2026-08-26 por el ADR 0021, posterior a este reporte. `demo.space-os.io` se conserva: es la demostración de las instancias hijas. No queda ninguna acción de este tipo pendiente para cerrar la Fase 4.

~~Borrar el registro A de `demo.space-os.io` en Cloudflare. Es una acción de navegador, sin ningún servidor de por medio.~~

Lo que decía este apartado, y por qué ya no aplica

Advertía de que mientras `demo.space-os.io` apuntase a la máquina de julio, esa máquina seguiría sirviendo un sitio público con cinco organizaciones dentro — y lo planteaba como el riesgo que había que cerrar.

El hecho sigue siendo cierto; su tratamiento cambió. Con el ADR 0021 el nombre se conserva a propósito, así que eso pasa a ser una condición aceptada y registrada, no una tarea pendiente. El criterio 3 de F4.5 queda retirado, no incumplido.

7.2 · El bloqueo, y a quién le toca

Bloqueo	Qué detiene	Abierto desde
TH-P4 · el registry de imágenes	F3.5, F3.6, el criterio 4 de F4.5, y F5.5, F5.6, F5.7 de la fase siguiente	17 de agosto

Son dos variables de configuración en el repositorio de GitHub. Sin ellas el flujo de publicación no puede ni identificarse contra el almacén, así que **nunca se ha publicado una imagen** y no existe canal del que las instancias puedan jalar su versión. Es el punto de mayor apalancamiento del plan entero.

7.3 · Tareas abiertas que este corte no cierra

- El proceso del servidor padre corre como root. La demostración ya no; el

padre sí. Lo coherente, y así lo deja escrito el ADR 0019, es que ambos pasen a servicio del sistema con usuario propio.

- El valor por omisión de la conexión a base de datos. Una instancia sin esa

configuración **debería negarse a arrancar**. Peor: si alguien levanta el entorno de desarrollo en esa máquina, la base de desarrollo sí responde y la aplicación hablaría con ella creyendo que es la suya. Es el fallo que ya mordió al servidor padre, y en la Fase 5 se repetiría en cada servidor nuevo.

- El humo del procedimiento de instalación tiene que tocar la base (§5).
- Los códigos de recuperación del Dueño. Decididos el 20/08, no construidos.

Hoy no hay segunda vía si alguien pierde su cuenta de Google.

- Una migración fallida deja la base sin recobro (defecto D4).
- Modo de arranque de la aplicación: el padre y la demostración arrancan igual,

con un aviso del framework. Condición preexistente; se decide para los dos a la vez.

8 · Nota de integridad: desfases corregidos al cerrar las fases

Al levantar este reporte aparecieron desfases entre lo que decían los documentos y lo que era cierto. Los cuatro primeros quedaron **corregidos** en el mismo commit que cierra las fases; los dos últimos **se declaran**, porque no son un error que arreglar.

1.  El expediente pedía dos pasos que el ADR 0020 anuló — el certificado de `demo.space-os.io` y mover su registro A. Lo llamativo es que el ADR los eliminó el mismo día en que esa sección se escribió, mientras el resto del documento sí se actualizaba.
2.  El tablero de ejecución iba cinco días por detrás: marcaba F4.2 y F4.4 como «ensayada en local» del 19/08 con su parte real medida contra el servidor desde el 25/08.
3.  No había entrada de bitácora del 26/08 para el ADR 0020, y esa decisión cambia lo que ve un cliente. Escrita en lenguaje llano.
4.  Cinco notas internas daban por vivo el nombre de la demostración, o por perdido un acceso que nunca se perdió. Corregidas con aviso fechado y sin reescribir su cuerpo: eran correctas en su fecha.
5.  La rama no está empujada al remoto. Es estado, no defecto: se empuja cuando corresponda.
6.  Nada está fusionado a `main`, y es deliberado: la rama principal tiene 66 migraciones y esta 73. El modelo de instancias se aísla a propósito hasta que el canal esté probado.

El patrón importa más que los casos, y ya va por la quinta vez

Los cuatro desfases corregidos son el mismo defecto: un documento que sobrevive a la decisión que lo invalida. No es descuido — es que quien toma la decisión no suele ser quien mantiene la fila que la describe. A la quinta repetición en esta rama deja de ser un accidente y pasa a ser algo que el proceso tiene que resolver.

Preparado el 2026-08-26 sobre la rama `feat/servidor-padre-instancias`. Ninguna tarea pendiente se ha ejecutado contra un servidor sin decirlo.